Network Swiss Army Knife (NSAK)

Bachelor Thesis – Integrating Agentic AI for Network Reconnaissance

June 7, 2026

Lukas von Allmen (vonal3) and Frank Gauss (gausf1) | Bern University of Applied Sciences

Introduction

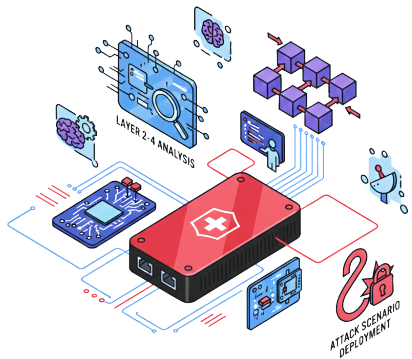
Starting Point: The NSAK Framework

Established in the predecessor project (Project 2)

A modular, open-source network security framework

Built around four reusable resource types:

- **Device** *execution host at a network position*
- **Environment** *target network topology*
- **Scenario** *a red/blue team use case*
- **Drill** *reusable building block*



Hypothesis and Research Questions

From static scenarios to agentic AI

Hypothesis

Integrating **agentic AI** into NSAK improves **adaptability, flexibility** and **automation** over static scenarios, while **lowering the operator's required expertise** to interpret results and plan follow-up steps.

Hypothesis and Research Questions

From static scenarios to agentic AI

Hypothesis

Integrating **agentic AI** into NSAK improves **adaptability, flexibility** and **automation** over static scenarios, while **lowering the operator's required expertise** to interpret results and plan follow-up steps.

RQ1 How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?

RQ2 To what extent can agentic AI **support operators** during interactive reconnaissance?

RQ3 To what extent is the implementation **suitable for real-world** deployment?

Background and Concepts

NSAK as AI Harness

The framework as the agent's execution environment

The agent does not run standalone –
NSAK is the harness around it:

- **Execution host** Device at a strategic network position, isolated in a container

NSAK as AI Harness

The framework as the agent's execution environment

The agent does not run standalone –
NSAK is the harness around it:

- **Execution host** Device at a strategic network position, isolated in a container
- **Drills as tools** every drill auto-wrapped as a type-safe LangChain tool

NSAK as AI Harness

The framework as the agent's execution environment

The agent does not run standalone –
NSAK is the harness around it:

- **Execution host** Device at a strategic network position, isolated in a container
- **Drills as tools** every drill auto-wrapped as a type-safe LangChain tool
- **Scenarios as entrypoints** AI scenarios run interactive, multi and single agent

NSAK as AI Harness

The framework as the agent's execution environment

The agent does not run standalone –
NSAK is the harness around it:

- **Execution host** Device at a strategic network position, isolated in a container
- **Drills as tools** every drill auto-wrapped as a type-safe LangChain tool
- **Scenarios as entrypoints** AI scenarios run interactive, multi and single agent
- **Configuration** provider, model and credentials via runtime config

NSAK as AI Harness

The framework as the agent's execution environment

The agent does not run standalone –
NSAK is the harness around it:

- **Execution host** Device at a strategic network position, isolated in a container
- **Drills as tools** every drill auto-wrapped as a type-safe LangChain tool
- **Scenarios as entrypoints** AI scenarios run interactive, multi and single agent
- **Configuration** provider, model and credentials via runtime config
- **Safety & observability** kill switch, structured results, Grafana / Loki logging

NSAK as AI Harness

The framework as the agent's execution environment

The agent does not run standalone – NSAK is the harness around it:

- **Execution host** Device at a strategic network position, isolated in a container
- **Drills as tools** every drill auto-wrapped as a type-safe LangChain tool
- **Scenarios as entrypoints** AI scenarios run interactive, multi and single agent
- **Configuration** provider, model and credentials via runtime config
- **Safety & observability** kill switch, structured results, Grafana / Loki logging

Reconnaissance in the attack chain

Host discovery → port scanning → service enumeration → (credential access / lateral movement)

The agent reuses NSAK building blocks instead of ad-hoc scripts.

Implementation

Agent Capabilities and Safety

Tools

LangChain tools available to the agent

- `host_configuration` situational awareness (interfaces, target/mgmt)
- `cli_tool` arbitrary shell – nmap, tshark, …
- Drills as tools `discover_hosts`, `port_scan` auto-wrapped
- `send_email` reports to a fixed recipient
- `human_interaction_hook` pause & ask the operator
- MCP external draw.io diagram server

Implemented Scenarios

One static baseline, three AI scenarios

Static Reference (baseline)

- Fixed drill chain: `arp-scan` → `nmap -sV` → NSE scripts
- Deterministic, structured result, predictable errors

Implemented Scenarios

One static baseline, three AI scenarios

Static Reference (baseline)

- Fixed drill chain: `arp-scan` → `nmap -sV` → NSE scripts
- Deterministic, structured result, predictable errors

AI Agent

- single and multi-agent
- unstructured and structured output
- interactive with human interaction hook.
- model independent

Implemented Scenarios

One static baseline, three AI scenarios

Static Reference (baseline)

- Fixed drill chain: `arp-scan` → `nmap -sV` → NSE scripts
- Deterministic, structured result, predictable errors

AI Agent

- single and multi-agent
- unstructured and structured output
- interactive with human interaction hook.
- model independent

AI Reconnaissance (main)

- 5-step prompt; **agent** picks nmap flags & interprets results
- Reports findings + vulnerability indicators

How We Integrated the LLM

NSAK as the harness – the agent only reasons, the framework acts

- **Capabilities become tools** each drill is exposed to the agent
- **Provider-agnostic** local (Ollama) and hosted models behind one interface, swapped via config
- **Cross-cutting middleware** one place to add logging, safety and context control around *every* agent

How We Integrated the LLM

NSAK as the harness – the agent only reasons, the framework acts

- **Capabilities become tools** each drill is exposed to the agent
- **Provider-agnostic** local (Ollama) and hosted models behind one interface, swapped via config
- **Cross-cutting middleware** one place to add logging, safety and context control around *every* agent

How We Integrated the LLM

NSAK as the harness – the agent only reasons, the framework acts

- **Capabilities become tools** each drill is exposed to the agent
- **Provider-agnostic** local (Ollama) and hosted models behind one interface, swapped via config
- **Cross-cutting middleware** one place to add logging, safety and context control around *every* agent

How We Integrated the LLM

NSAK as the harness – the agent only reasons, the framework acts

- **Capabilities become tools** each drill is exposed to the agent
- **Provider-agnostic** local (Ollama) and hosted models behind one interface, swapped via config
- **Cross-cutting middleware** one place to add logging, safety and context control around *every* agent

One abstraction, three modes

The same agent powers **single-agent**, **multi-agent** and **interactive** scenarios.

The remaining slides show the three concepts we had to solve to make this reliable.

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Our solution

- Split the task into **discovery** → **enumeration** → **assessment**
- Each phase is a **fresh agent** with its own, small context
- We pass on only the **distilled result**, not the whole history

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

<5-> ⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Our solution

- Split the task into **discovery** → **enumeration** → **assessment**
- Each phase is a **fresh agent** with its own, small context
- We pass on only the **distilled result**, not the whole history

ℹ Trade-off: rescues small models, but adds orchestration overhead that frontier models don't need.

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

⚠ The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

⚠ The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Our solution

- The agent **pauses** before sensitive actions and asks for a decision
- Shell commands → **approve** before they run
- Sending a report → **approve / reject**
- Open questions → operator **responds** in free text

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Our solution

- The agent **pauses** before sensitive actions and asks for a decision
- Shell commands → **approve** before they run
- Sending a report → **approve / reject**
- Open questions → operator **responds** in free text

After the decision the agent resumes exactly where it stopped – no restart, no lost progress.

Answering the Research Questions

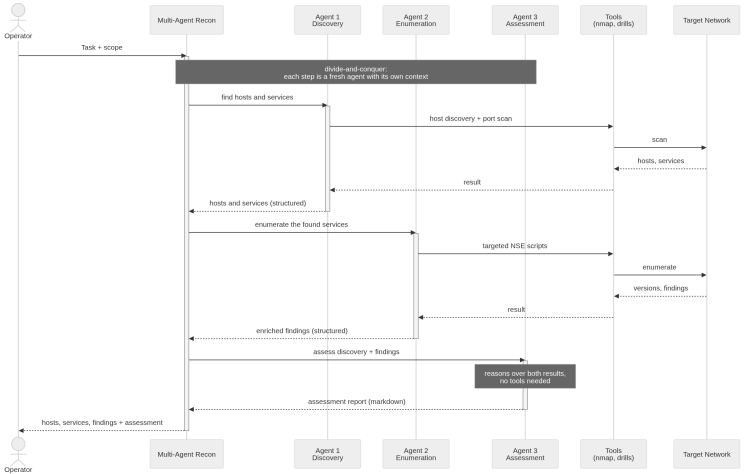
RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **interpretation** (credentials, banner reasoning, severity scoring).

RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. `--interactive` preserves human control.

RQ3 **Deployment:** Scales to ∼35 hosts, highlights key risks. **Requires supervision** (reliability limits).

Multi-Agent Reconnaissance Pipeline

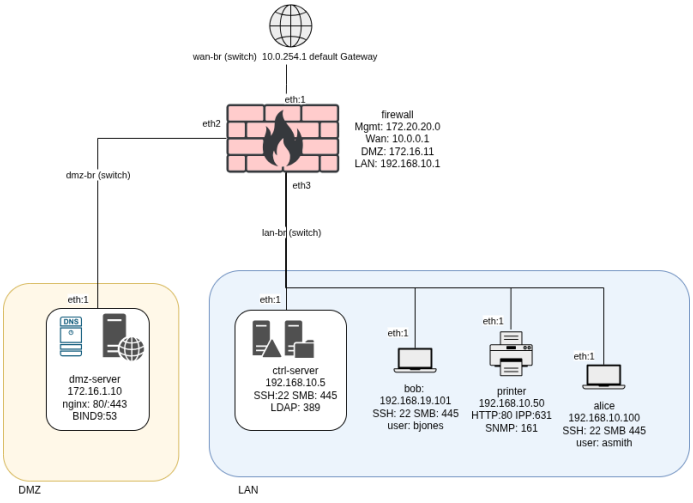
Discovery → enumeration → assessment, each a fresh agent



Evaluation Setup

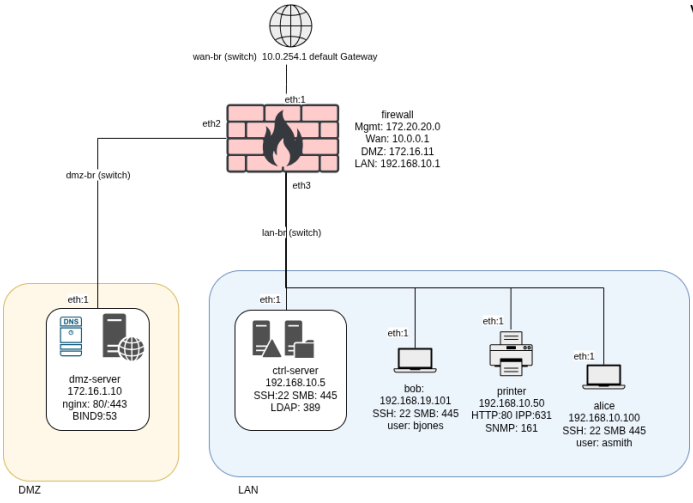
Reproducible Test Environment

A virtual enterprise network built with Containerlab



Reproducible Test Environment

A virtual enterprise network built with Containerlab

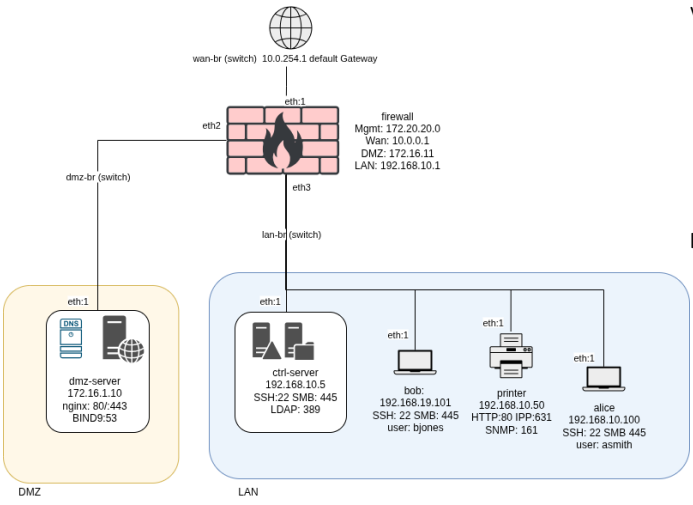


Why Containerlab?

- Reproducible topology from one config file
- Multi-interface nodes & fast teardown
- Clean state per experiment

Reproducible Test Environment

A virtual enterprise network built with Containerlab



Why Containerlab?

- Reproducible topology from one config file
- Multi-interface nodes & fast teardown
- Clean state per experiment

Intentional vulnerabilities

- LDAP anonymous bind + cleartext creds (`Password123!`)
- SMB signing not required; public share
- SSH banners leak org name
- DMZ: DNS zone transfer, `/secret.txt`

Evaluation Methodology

Models, environments and criteria

Three models

- `claude-opus-4-7` frontier (Anthropic API)
- `gpt-oss:120b` open-weight, BFH-hosted
- `qwen3:30b` local, self-hosted (Ollama)

Three environments

- Virtual (Containerlab) – reproducible
- BFH Network Security Lab – real network

- **Quantitative** duration, token usage, hosts / services / findings
- **Qualitative** correctness & hallucination (1–10 scale), consistency

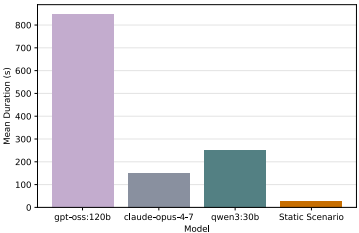
Benchmark suite

`BenchmarkManager` runs each scenario n times, resets state between runs, and aggregates a `BenchmarkReport` (JSON + Markdown) – automating the quantitative metrics.

Results

Containerlab – Multi-Agent Benchmark

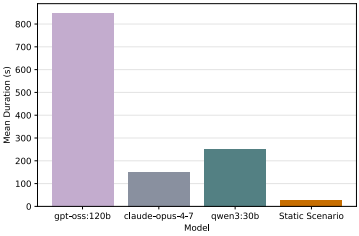
Multi-agent configuration ($n = 10$ each)



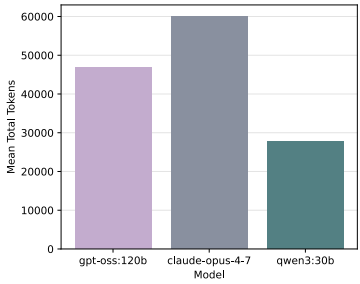
Duration

Containerlab – Multi-Agent Benchmark

Multi-agent configuration ($n = 10$ each)



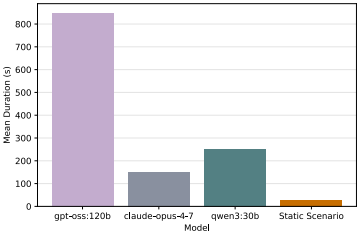
Duration



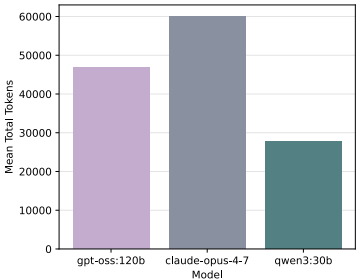
Token usage

Containerlab – Multi-Agent Benchmark

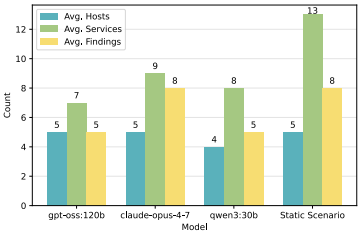
Multi-agent configuration ($n = 10$ each)



Duration



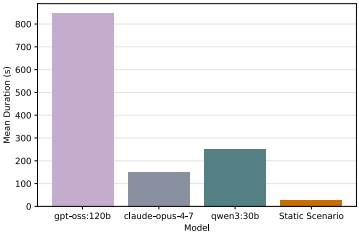
Token usage



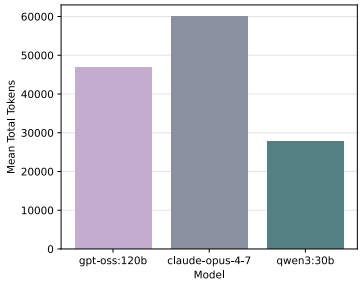
Discoveries

Containerlab – Multi-Agent Benchmark

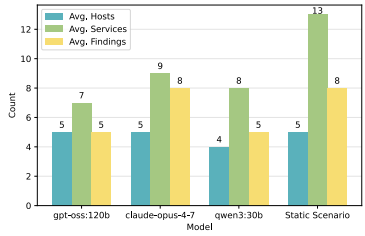
Multi-agent configuration ($n = 10$ each)



Duration



Token usage

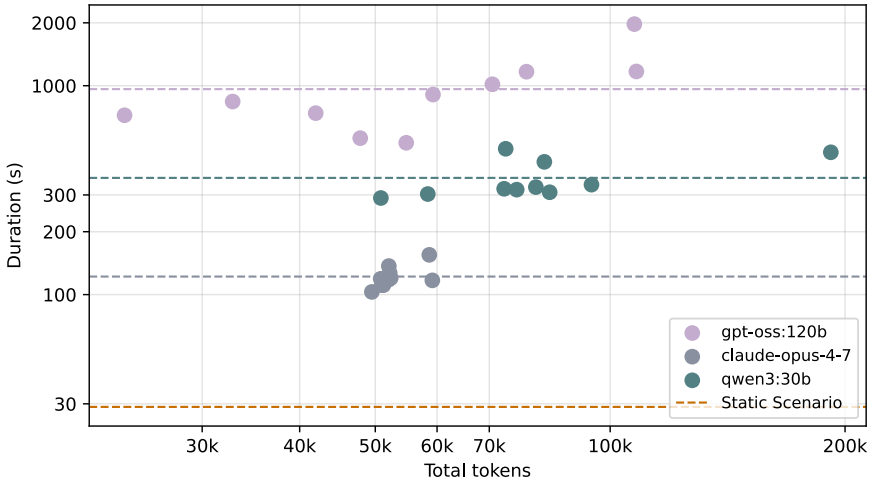


Discoveries

- Decomposition **helps the smaller models**: `qwen3:30b` drops tokens **–68%** (86k → 28k) and duration **–30%**.
- `claude-opus-4-7` keeps identical discovery quality (slight +13% token overhead); static baseline constant at **29 s**.

Containerlab – Token vs Duration plot

Multi-agent model comparison



Containerlab – Qualitative Results

The discriminator: extracting the LDAP cleartext credentials

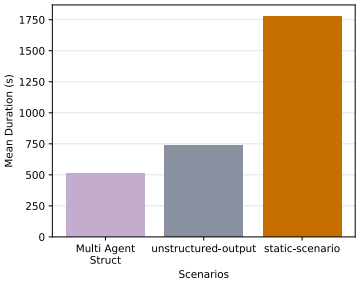
- **claude-opus-4-7** extracts the credentials in **every run** + correct attack-chain reasoning (reuse → lateral movement). Wrongly calls the printer a honeypot.
- **gpt-oss:120b** extracts creds in ∼half the runs; **fabricates CVE IDs**.
- **qwen3:30b** discovers hosts/services but rates LDAP “Low” risk in 9/10 runs – **misses the critical finding**.
- **Static** never performs the anonymous bind – credentials stay unextracted despite full port coverage.

- Only the **frontier model** reaches the critical cleartext credentials.
- **SNMP** probed by nobody (only TCP sweeps); DMZ host outside the target subnet – unreached.

Trade-off: static = breadth & determinism; AI = **interpretive depth** (severity-rated assessment, next steps).

BFH Network Security Lab – Benchmark

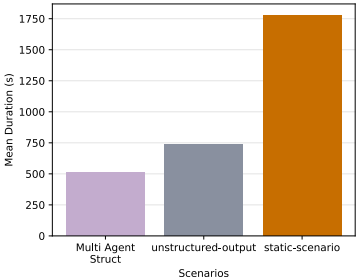
A real network ≈7× larger (≈35 live hosts)



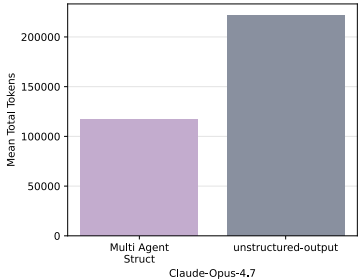
Duration

BFH Network Security Lab – Benchmark

A real network ≈7× larger (≈35 live hosts)



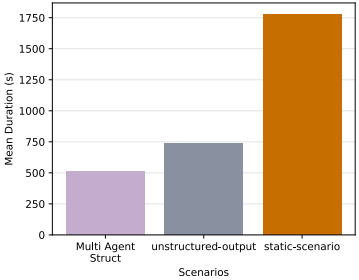
Duration



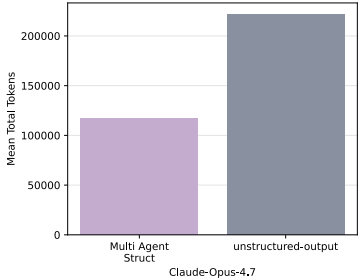
Token usage

BFH Network Security Lab – Benchmark

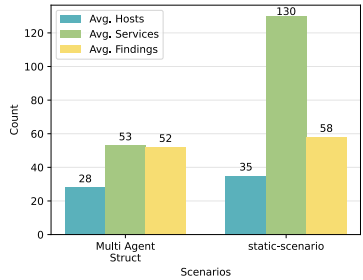
A real network ≈7× larger (≈35 live hosts)



Duration



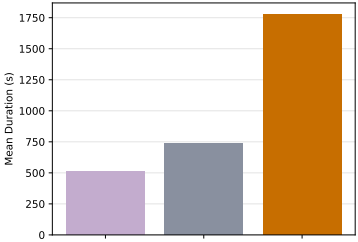
Token usage



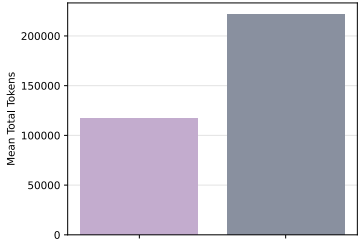
Discoveries

BFH Network Security Lab – Benchmark

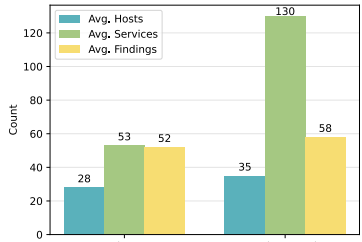
A real network ≈7× larger (≈35 live hosts)



Duration



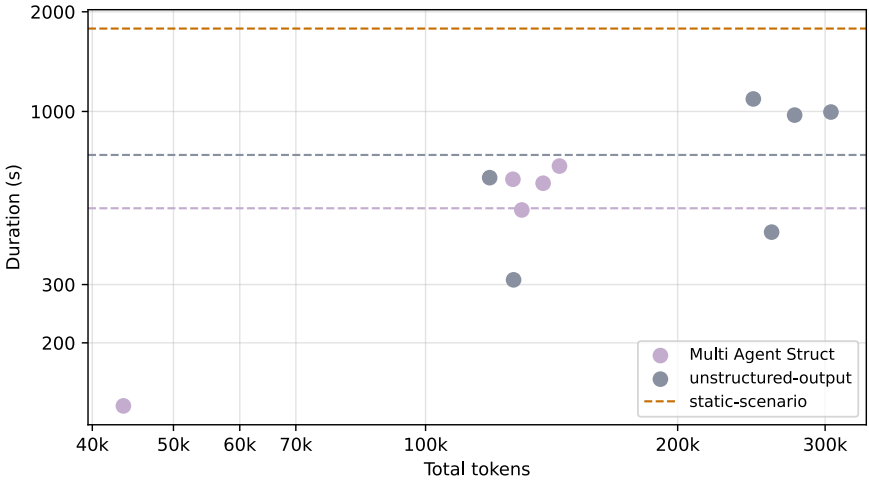
Token usage



Discoveries

- **Frontier model scales**: 28 hosts, 53 services, 52 findings; surfaces all 6 high-severity risks.
- **Reliability is the bottleneck**: low success (multi-agent **5/27**, unstructured **6/109**); **`qwen3:8b`** makes **zero tool calls**.
- Static scenario is exhaustive but slow (**1780 s** for the whole network).

Network Security Lab – Token vs Duration plot



Discoveries

Discussion and Conclusion

Answering the Research Questions

RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

Answering the Research Questions

RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. `--interactive` Operator stays in the loop and control.

Answering the Research Questions

RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. `--interactive` Operator stays in the loop and control.

RQ3 **Deployment:** Scales to ∼35 hosts, highlights key risks. **Requires supervision** (reliability limits).

Key Findings

Takeaways across both environments

What works

- Even smaller models can autonomously run **standard red-team reconnaissance**
- **Multi-agent decomposition** helps smaller models (–68% tokens for `qwen3:30b`)
- AI delivers **depth**: assessment, prioritized findings, remediation

Key Findings

Takeaways across both environments

What works

- Even smaller models can autonomously run **standard red-team reconnaissance**
- **Multi-agent decomposition** helps smaller models (–68% tokens for `qwen3:30b`)
- AI delivers **depth**: assessment, prioritized findings, remediation

What limits it

- **High failure rate** – best-case subset
- **Hallucinations**: fabricated CVEs, honeypots
- **Model tier matters**: `qwen3:8b` unusable
- Manual qualitative scoring

Recommendations and Future Work

Recommendations

- **Match model tier to task** – frontier model for autonomous recon
- **Multi-agent** for smaller / self-hosted models
- **Hybrid pipeline**: static breadth + AI depth (+ RAG)
- **Human-in-the-loop** + retry on incomplete runs

Future work

- **Dynamic tool calling** – expose only relevant tools
- **Guardrails & sandboxing** – scope restriction for production
- Robust **HITL middleware**
- **Broader attack-chain** coverage beyond reconnaissance

Keine Ahnung warum es das diagram nicht rendert ?????

Conclusion

Agentic AI complements deterministic enumeration

- ✔ Using NSAK directly as an **agentic harness** proved effective – the agent autonomously and interactively executes network discovery via LangChain tools.
- ✔ Scripted scenarios excel in **speed and reproducibility**; the agent adds **reasoning and assessment**, limited by model capability.
- ✔ **Multi-agent decomposition** improves smaller models by reducing context complexity and increasing consistency.
- ⚠ Hallucinations remain a limitation – **supervised operation with a human reviewer** is the proposed deployment model.

LLMs make cybersecurity use cases more accessible and cost-effective – with a human in the loop.

Demo: Agentic Reconnaissance in Action

Questions?